Guilherme Henrique Alves Rodrigues

Gerenciamento de desempenho em Redes Definidas por Software utilizando Agente Baseado em Meta

São Paulo – Brasil

2019

Guilherme Henrique Alves Rodrigues

Gerenciamento de desempenho em Redes Definidas por Software utilizando Agente Baseado em Meta

Monografia apresentada na disciplina Trabalho de Conclusão de Curso, como parte dos requisitos para obtenção do título de Bacharel em Ciência da Computação.

Centro Universitário Senac - Santo Amaro

Bacharelado em Ciência da Computação

Orientador: Prof. Dr. Denis Gabos

São Paulo – Brasil

2019

Dedicatória

Ao meu Orientador Prof. Dr. Denis e minha

esposa Dianna que me iluminaram nessa trajetória.

Resumo

As redes, atualmente, emergem em mudanças cada vez mais rápidas, com as novas integrações de tecnologias em informática e comunicação (XIAO, 2008), uma documentação é proposta para auxiliar equipes na análise em projetos complexos de redes, o que levará a respostas mais rápidas aos problemas (APPLEBY,2002). Por documentação entendemos todo o conjunto de documentos que descrevem a rede (DAUTOVIC, 2008).

Incluir Inteligência Artificial (I.A.) para tomada de decisões dentro da rede tem como intuito melhorar o desempenho, pois faz a antecipação de eventuais problemas (SENDRA, 2017). Neste trabalho foi utilizado o Mininet para simular a Rede Definida por *Software* (SDN) e foi proposto utilizar Agente Inteligente para definir os fluxos de rede. Ao utilizar a Rede Definida por *Software* (SDN) em conjunto com o *OpenFlow* se obtém a flexibilidade necessária, pois pode-se configurar os equipamentos da rede a partir de um único ponto. Para o uso do Agente Inteligente, é necessário efetuar a coleta dos fluxos, essa coleta foi feita pelo controlador *OpenDayLight* (ODL) e pelo comando via *Linux* SSH (*Secure Shell*). Com a compreensão do estado da rede pelo Agente Inteligente, é possível alterá-la, manter um histórico de alterações e analisar suas consequências.

Palavras-chaves: Rede Definida Por Software, OpenFlow, Agente Inteligente

Abstract

Networks nowadays emerge in ever faster changes with new integrations of information and communication technologies (XIAO, 2008), documentation is proposed to assist teams in analyzing complex network projects, which will lead to faster responses to problems (APPLEBY, 2002). By documentation we mean the whole set of documents that describe the network (DAUTOVIC, 2008).

Including Artificial Intelligence (I.A.) for decision making within the network is intended to improve performance, as it anticipates any problems (SENDRA, 2017). In this work the Mininet was used to simulate the Software Defined Network (SDN) and it was proposed to use Intelligent Agent to define the network flows. Using the Software Defined Network (SDN) in conjunction with OpenFlow gives you the necessary flexibility because you can configure your network equipment from a single point. To use the Intelligent Agent, it is necessary to collect the flows, this collection was done by the OpenDayLight (ODL) controller and by the command via Linux SSH (Secure Shell). By understanding the status of the network by the Intelligent Agent, you can change it, keep track of changes, and analyze its consequences.

Palavras-chaves: Software Defined Network, OpenFlow, agent intelligent

Lista de ilustrações

Figura 1 – Redes Definidas por Software 11
Figura 2 – Componentes do OpenDayLight 13
Figura 3 – Arquitetura do OpenFlow 14
Figura 4 – Comunicação via Openflow 15
Figura 5 – Um simples exemplo de uma rede Mininet 16
Figura 6 – Open vSwitch 17
Figura 7 – Agente Inteligente 19
Figura 8 – Árvore de Decisão - Entrada de Atributos 20
Figura 9 – Árvore de Decisão - Restaurante 20
Figura 10 – Algoritmo de Prim 21
Figura 11 – Topologia Proposta 24
Figura 12 – Agente Baseado em Metas 26
Figura 13 – Topologia criada no MiniEdit 27
Figura 14 – Comando Dump do Mininet 27
Figura 15 – Acessando o mininet por SSH e executando o MiniEdit 28

Sumário

Lista de ilustrações **5**

1 INTRODUÇÃO **8**
1.1 Motivação **8**
1.2 Objetivo **8**
1.3 Objetivos Específicos **8**
1.4 Estrutura do Trabalho **9**

2 METODOLOGIA **10**

3 REVISÃO BIBLIOGRÁFICA **11**
3.1 Rede Definida por *Software* (SDN) **11**
3.1.1 Controlador OpenDayLight (ODL) 12
3.2 OpenFlow **13**
3.3 Simulador de rede **15**
3.4 Open vSwitch (OVS) **16**
3.5 Secure shell (SSH) **17**
3.6 Iperf **17**
3.7 Traffic Control (TC) **17**
3.8 Coleta de dados via API **18**
3.9 Agente Inteligente **18**
3.10 Algoritmos **18**
3.10.1 Árvore de Decisão 19
3.10.2 Algoritmo de Prim 20
3.11 Trabalhos Relacionados **21**
3.11.1 *A Hybrid Intelligent Approach for Optimising Software-Defined Networks Performance* 21
3.11.2 *Machine Learning in Software Defined Networks: Data Collection and Traffic Classification* 22
3.11.3 *Including Artificial Intelligence in a Routing Protocol Using Software Defined Networks* 22

4 DESENVOLVIMENTO **23**
4.1 Arquitetura do Sistema **23**
4.2 Testes **29**
4.2.1 Teste do Agente Inteligente 30

4.2.2 Teste Controlador OpenDayLight 33
4.3 Avaliação e conclusões **34**
4.3.1 Trabalhos Futuros 34
4.4 Conclusão **35**

5 REFERÊNCIAS 36

1 Introdução

As redes, atualmente, emergem em mudanças cada vez mais rápidas, com as novas integrações de tecnologias em informática e comunicação (XIAO, 2008). O ideal é que toda rede tenha a sua documentação atualizada, pois uma documentação é proposta para auxiliar equipes na análise em projetos complexos de redes, o que levará a respostas mais rápidas aos problemas (APPLEBY,2002). Por documentação entendemos todo o conjunto de documentos que descrevem a rede (DAUTOVIC, 2008).

Incluir Inteligência Artificial (I.A.) para tomada de decisões dentro da rede tem como intuito melhorar o desempenho, pois faz a antecipação de eventuais problemas (SENDRA, 2017). Uma rede que aprenda a reconhecer o seu ambiente e antecipe qualquer problema se reconfigurando reduz a quantidade de paradas, o que evita que serviços importantes parem, como a Bolsa de Valores ou em um banco. Para mudanças rápidas e flexibilidade foi utilizada a Rede Definida por *Software* (SDN), onde as configurações são efetuadas por um ponto único, que pode configurar equipamentos que atuem como comutadores na rede.

1.1 Motivação

As redes estão em constante mudanças, a arquitetura de rede deve ter flexibilidade para permitir a expansão do uso sem queda de desempenho. Essa flexibilidade precisa ser inteligente para garantir que serviços não parem, este trabalho reune a flexibilidade da uma Rede Definida por *Software* com a Inteligencia Artificial para melhorar o desempenho nos fluxos dos dados.

1.2 Objetivo

O objetivo deste trabalho é manter o fluxo contínuo dos dados em uma Rede Definida por *Software* virtualizada, fazendo uso de inteligência Artificial, a fim de utilizar rotas que evitem a parada do fluxo dos dados, assim, mantendo os níveis de latência, *jitter*, perda de pacotes e banda de forma que mantenha o desempenho da rede.

1.3 Objetivos Específicos

- Implementar uma rede SDN no Mininet;
- Implementar o protocolo *OpenFlow* na rede SDN;

- Implementar o *Open vSwitch* para serem os comutadores de rede;
- Implementar o *Iperf* para efetuar a coleta de dados da rede SDN;
- Implementar a coleta de dados via API do Mininet;
- Implementar Árvore de Classificação no Agente Inteligente utilizando os dados coletados;
- Implementar o Algoritmo de Prim para remover *loop* na rede SDN;
- Implementar o utilitário de linha de comando *Traffic Control* (TC) para gerar perda de desempenho na rede para testes.

1.4 Estrutura do Trabalho

Capítulo 2 - Metodologia, onde é descrito o processo de criação deste trabalho.

Capítulo 3 - Revisão Bibliográfica, onde é descrito todos os itens necessários para compreensão do trabalho.

Capítulo 4 - Desenvolvimento, onde é descrito todo o processo de criação, testes e avaliações.

Capítulo 5 - Conclusão

2 Metodologia

Para obter um melhor desempenho na Rede Definida por *Software*, será utilizado um Agente Inteligente para decidir qual é a melhor rota para enviar o fluxo de dados.

Como demonstrado por (SABEEH, 2016) e (SENDRA, 2017), a Inteligência Artificial tem potencial para resolver problemas de rede, assim como (SENDRA, 2017) este trabalho utiliza Agente Inteligente.

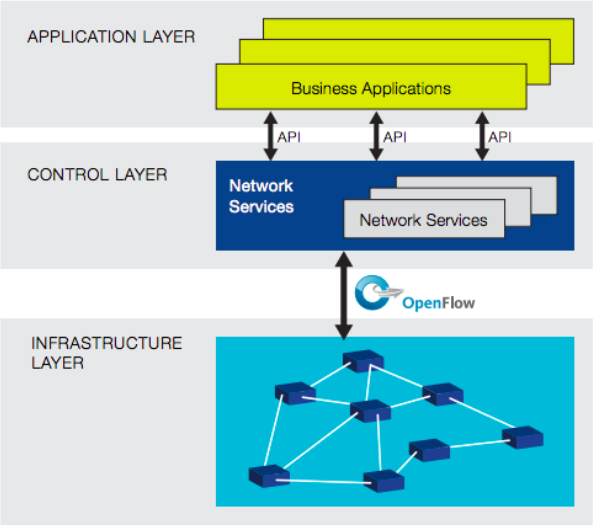
A Rede Definida por *Software* é simulada no *Mininet* e utiliza o protocolo *OpenFlow* para se comunicar com os comutadores de rede os *Open vSwitches*, que serão configurados e gerenciados pelo Agente Inteligente, por meio de API.

3 Revisão Bibliográfica

3.1 Rede Definida por Software (SDN)

Segundo JUNIOR (2014), as Redes Definidas por *Software* tem um novo paradigma de rede, onde o controle da rede é feito via *software* centralizado, que controla a rede inteira. É uma nova arquitetura que fornece redes mais ágeis e rentáveis, composta por três camadas distintas, acessíveis através de API (*Application Programming Interface*) abertas (conforme figura 1):

Figura 1 – Redes Definidas por Software



Fonte: SDN Security Considerations in the Data Center ONF Solution Brief, October 8, 2013, página 3

- Aplicação: composta por aplicações de negócio do usuário final que utiliza os serviços de comunicações da Rede Definida por *Software*. O que separa a Camada de Aplicação e a Camada de Controle é a *northbound* API, que é por onde são passados os dados.
- Controle: supervisiona o comportamento das rotas via interface aberta e permite o controle desse fluxo de dados.
- Infraestrutura: contém os dispositivos que fazem a comutação e encaminhamento de pacotes.

Segundo JUNIOR (2014), as Redes Definidas por *Software* tem três características principais:

- Inteligência Logicamente Centralizada: Em uma arquitetura SDN, o OpenFlow faz encaminhamento do controle da rede de modo distribuído. Como a inteligência é centralizada a tomada de decisão é simplificada por ter visão global da rede. Diferente de hoje, que os nós das redes não tem conhecimento do estado da rede.

- Programabilidade: Redes SDN são controladas por funcionalidades do *software*, que podem ser de terceiros ou criados pelos operadores de rede. Isso permite que o paradigma de gestão possa ser substituído pela automação. Ao fornecer API aberta para que as aplicações se comuniquem com a rede, as Redes Definidas Por *Software* disponibilizam inovação sem precedentes.

- Abstração: Em redes SDN, as aplicações que utilizam os serviços de SDN são abstraídos das tecnologias de rede subjacentes. Já os dispositivos são abstraídos da camada de controle, isso garante a portabilidade e futuro do investimento em serviços de rede.

  Para KREUTZ (2015), o encaminhamento de pacotes é simplificado pela separação da rede em plano de controle e plano de dados. Ao utilizar abstração de fluxo, permite unificar o comportamento de diferentes tipos de dispositivos de rede, o que permite uma flexibilidade sem precedentes, limitada apenas pela implementação da tabela de fluxo.

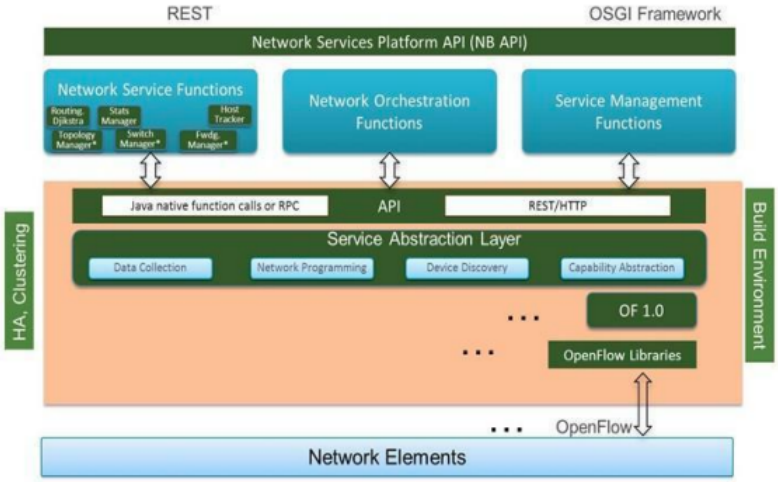
3.1.1 Controlador OpenDayLight (ODL)

O controlador *OpenDaylight* (ODL) é uma plataforma flexível para personalizar e automatizar redes de qualquer tamanho e escala. Com foco na programação de rede, ele foi projetado para soluções que abordam uma variedade de casos de uso em ambientes de redes (OPENDAYLIGHT).

OpenDayLight é um controlador de código aberto dos projetos colaborativos da *Linux Foundation*. É uma aplicação de software e, como uma *Java Virtual Machine* (JVM), pode ser executada em qualquer sistema operacional que suporte Java. Contém API para comunicação com outros *softwares* (SISOV, 2016).

O controlador como visto na figura 2 é um ponto estratégico, que utiliza o protocolo *OpenFlow* para se conectar e configurar dispositivos de rede como roteadores, *switches* e etc, podendo determinar a melhor rota disponível por exemplo. O controlador pode simplificar o gerenciamento da rede, por ter a capacidade de manipular todas as comunicações entre aplicativos e dispositivos, alterando os fluxos de rede para se adaptar às mudanças. O Controlador envia as informações aos *switches* e roteadores por APIs *southbound* e aos aplicativos de lógica de negócios por APIs do *northbound*. (SDX Central)

Figura 2 – Componentes do OpenDayLight



Fonte: Building a Software-Defined Networking System with OpenDaylight Controller, 2016, página 14.

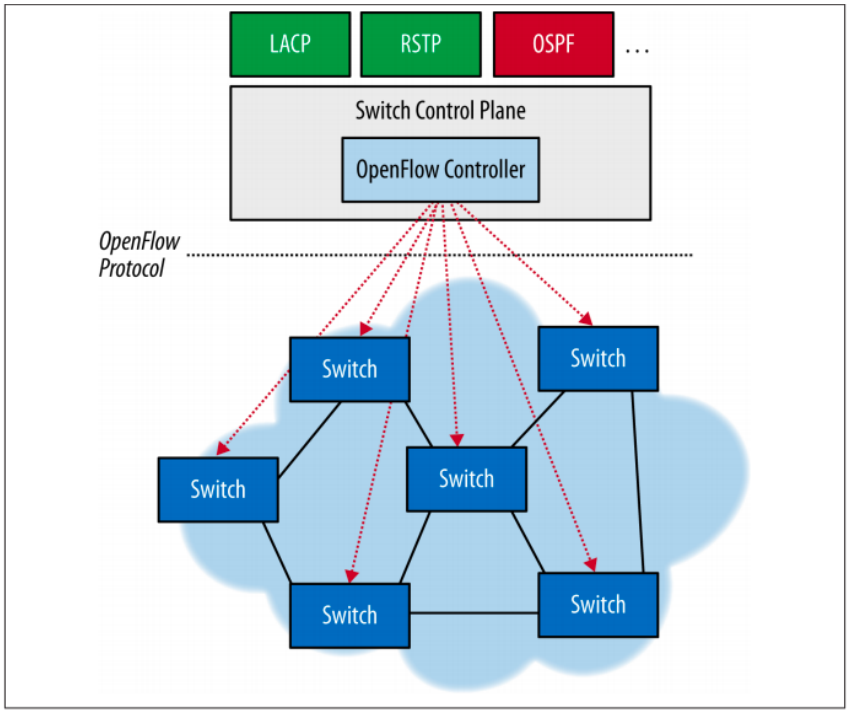
O *OpenDayLight* foi projetado usando uma estrutura baseada em relacionamentos consistentes entre modelos, mapeamentos padronizados e padrões que permitem a geração de modelos e, por extensão, geração de código/API a partir de modelos. Existem API REST *northbound* abertas disponíveis para uso. Estas são usadas por aplicativos da web que não residem no mesmo IP ou *hardware* que o controlador. (SISOV, 2016).

3.2 OpenFlow

OpenFlow é um protocolo aberto que define as ações de encaminhamento de pacotes, por exemplo, em *switches* e roteadores. O controlador faz o gerenciamento das regras e ações. A confiabilidade da comunicação entre os dispositivos de rede e o controlador é estabelecida ao utilizar o protocolo SSL (*Secure Socket Layer*). (JUNIOR, 2014)

Segundo JUNIOR (2014) o protocolo *OpenFlow* pode ser dividido em três características como pode ser visto na figura 3.

Figura 3 – Arquitetura do OpenFlow



Fonte: SDN - Software Defined Networks, O'REILLY, 2013, página 48.

1. A separação entre o plano de dados e o plano de controle, onde o plano de controle é gerenciado por um sistema controlador centralizado.

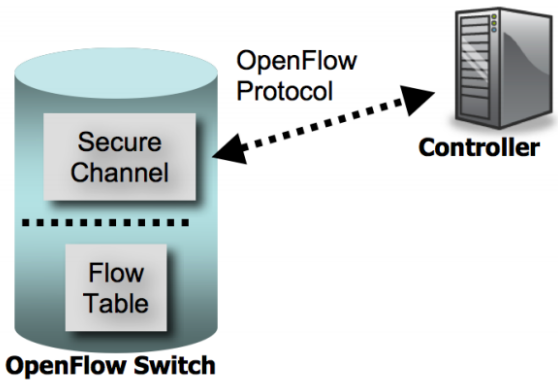
2. A padronização do protocolo entre o controlador e um agente na rede para instanciação.

3. Prover uma rede programável a partir de um ponto centralizado na rede via API.

O *OpenFlow* é um conjunto de protocolos e API e necessita de um programa para gerenciar o controlador, que fornece instruções sobre os fluxos dos dados (O'REILLY, 2013). Pela interface é possível adicionar ou remover entradas nas tabelas de fluxos dos *Open vSwitches*, isso facilita a criação de *softwares* para fazer o gerenciamento dos fluxos, incentivando a inovação na pesquisa de novos protocolos de roteamento por exemplo (FAUSTINI, 2016).

A comunicação entre o *switch* e o controlador como visto na figura 4 é por um canal seguro, por essa comunicação que o controlador efetua a configuração, gerencia e recebe os eventos do *switch* (ONF, 2009).

Figura 4 – Comunicação via Openflow



Fonte: OpenFlow Switch Specification, December 31, 2009, página 1

Para funcionar, o protocolo precisa que os *switches* na rede o suportem. Há dois tipos de *switches*: dedicados e habilitados. O dedicado precisa possuir uma tabela de fluxos para encaminhar os fluxos, caso não exista entrada compatível na tabela de fluxos, enviar pacotes para o controlador, adiciona a nova entrada na tabela e possibilidade de descartar pacotes. O Habilitado precisa suportar os mesmos itens do dedicado e poder utilizar regras de roteamentos tradicionais. (FAUSTINI, 2016)

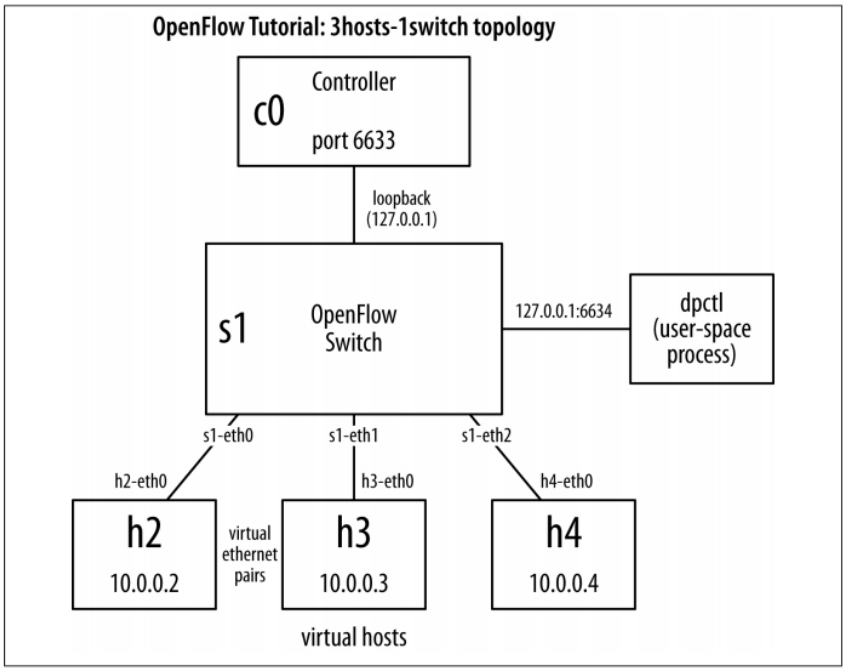
A interface é especificada na implementação, no entanto, todas as mensagens devem ser formatadas de acordo com o protocolo *OpenFlow*. (ONF, 2009)

3.3 Simulador de rede

O Mininet cria uma rede virtual realista em uma única máquina, que executa códigos reais de kernel de Sistemas Operacionais, *switches* e códigos de aplicações. (Mininet Team)

Na figura 5, é possível perceber que o controlador, o *switch* e os *hosts* estão na mesma máquina, e é utilizado o dpctl que é um utilitário de gerenciamento do *switch* que pode adicionar linhas na tabela de fluxo, consultar o status e alterar outras configurações.

Figura 5 – Um simples exemplo de uma rede Mininet



Fonte: SDN - Software Defined Networks, O'REILLY, 2013, página 87

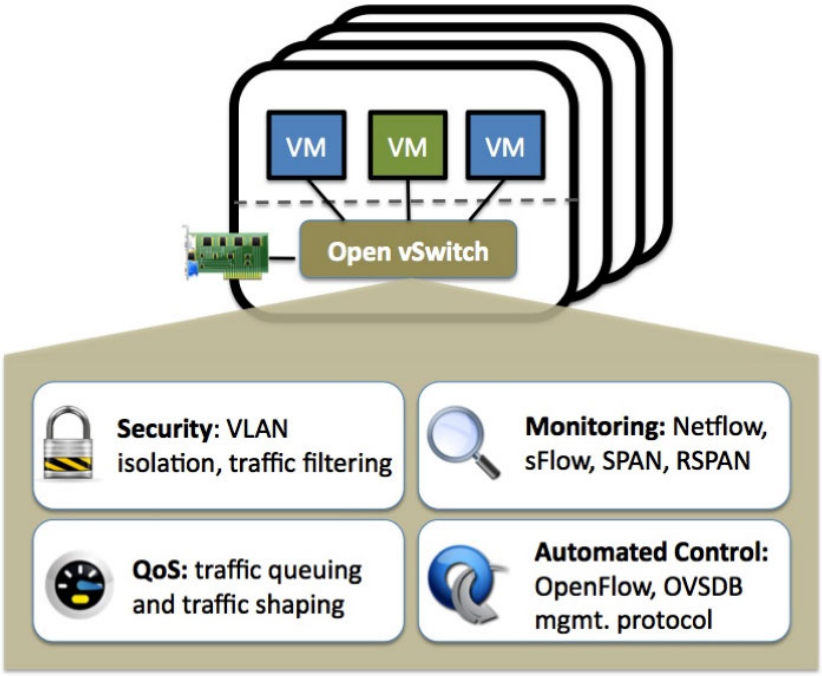
O NS3 (*Network Simulation* 3) também é um simulador de rede que dá suporte para redes sem fio, porém não oferece suporte ao TCP (*Transmission Control Protocol*), como o objetivo é simular uma rede real, foi utilizado o Mininet, mesmo possuindo suporte apenas para a rede cabeada (CONTERATO, 2013).

Para coleta de dados da rede o Mininet disponibilizada uma API e é possível utilizar diversos comandos como *ping*, *Iperf*, *wget*, *curl*, *netperf*, *netcat*, isso por conta da facilidade de executar praticamente qualquer programa *Linux*. Assim, é possível utilizar praticamente qualquer comando (MININET TEAM).

3.4 Open vSwitch (OVS)

O *Open vSwitch* representado na figura 6 é um *switch* virtual de *software* livre e tem uma tabela de fluxos que será responsável por encaminhar os pacotes; essa tabela pode ser alterada por comando. Possui as mesmas configurações do *Open Switch*, porém, o *Open vSwitch* é virtualizado, não necessitando de *hardware* para cada novo *switch*. Tem suporte ao *OpenFlow* utilizado nesse trabalho (SDX Central).

Figura 6 – Open vSwitch



Fonte: SDXCentral: What is Open vSwitch (OVS)?.

3.5 Secure shell (SSH)

Segundo OpenSSH (2019), SSH é um programa para efetuar login em uma máquina remota e executar comandos em uma máquina remota. Com comunicações criptografadas entre dois *hosts*.

Para se conectar via SSH é necessário o IP do *host* destino, usuário e senha, mas há outros métodos de provar a sua identidade (OpenSSH, 2019).

3.6 Iperf

O iperf é uma ferramenta para realizar medições de taxa de transferência de rede. Ele pode testar a taxa de transferência TCP ou UDP. Para executar um teste iperf, o usuário deve estabelecer um servidor (para descartar tráfego) e um cliente (para gerar tráfego) (DEBIAN, 2019).

3.7 Traffic Control (TC)

TC é utilizado para configurar controle de tráfego no Kernel do Linux. É possível adicionar latência, perda de pacotes, *jitter*, entre outros, em uma ou mais das placas de

rede de um dispositivo. O qdisc é a abreviação para disciplina de filas, quando o kernel precisa enviar um pacote ele envia para o qdisc. (DEBIAN, 2001)

3.8 Coleta de dados via API

Com a rede em execução no Mininet é possível que outro processo ou até mesmo outro computador consiga controlar a rede remotamente, basta utilizar o util/m no comando via SSH. Assim é possível executar os comandos na rede Mininet e coletar as informações necessárias (MININET TEAM).

3.9 Agentes Inteligentes

Segundo JENNINGS (1998), agentes são como um sistema computacional, localizados em um ambiente, sendo capazes de tomar uma ação autônoma e flexível para atingir seus objetivos.

A inteligência pode ser definida pela capacidade de aprender, de se adaptar, de melhorar o desempenho ao longo do tempo, de raciocinar, tomar decisões e de planejar. Tarefas complexas necessitam dessa inteligência, pois precisará colaborar com outros agentes e/ou usuários humanos. O agente inteligente pode ser estruturado no código do programa para aumentar sua inteligência, assim como as formas na vida que variam de forma, de unicelulares até formas de vida extremamente complexas como os humanos. (TSATSOULIS; SOH, 2000)

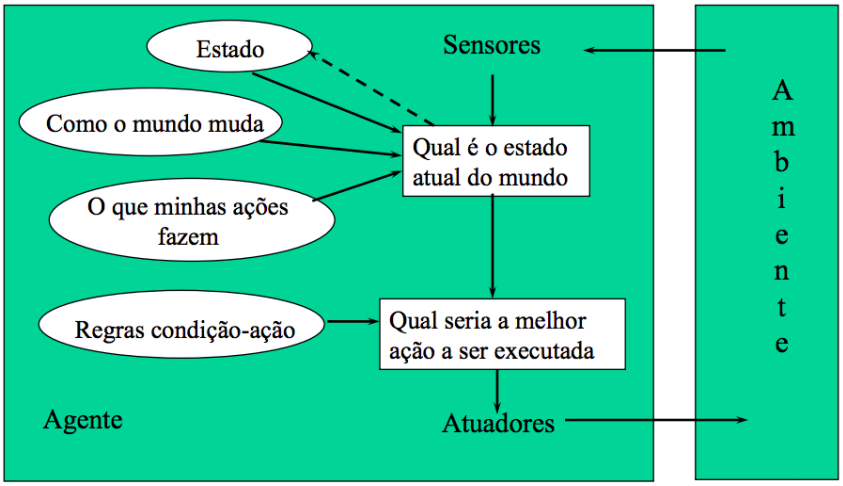
O Agente precisa ter, como mostrado na figura 7, itens como: percepção do seu ambiente (sensores, câmeras, GPS, etc), ações disponíveis, objetivos e Ambiente. Há medidas de desempenho para poder classificar o quão ótima está a solução. Agente Baseado em Metas são agentes que possuem visão parcial do mundo (ambiente completo), este agente coleta informações do seu ambiente, informações a respeito da evolução do mundo e os impactos de suas ações. (BARROS, 2006)

O comportamento de um Agente é dado pela função do Agente, onde f = P* -> A, sendo P* uma sequência de percepções (histórico do que já ocorreu) e A é uma ação. O Agente Baseado em Metas pode ter o seu objetivo direto, onde uma ação pode atingir a meta, ou indireto, necessitando de longas sequências para atingir a meta. As Metas descrevem ações desejáveis (LIMA, 2012).

3.10 Algoritmos

Neste projeto foram utilizados algoritmos, que fazem parte do Agente Inteligente.

Figura 7 – Agente Inteligente



Fonte: IME-USP, Introdução à Inteligência Artificial Agentes Inteligentes, BARROS, 2006

3.10.1 Árvore de Decisão

Uma árvore de decisão representa uma função que recebe como entrada um vetor de valores de atributos e retorna uma decisão, como mostrado na figura 8. Os valores de entrada e saída podem ser discretos ou contínuos. A árvore chega na sua decisão executando uma sequência de testes. Como mostrado na figura 9 cada nó interno na árvore corresponde a um teste de um dos atributos de entrada. Cada nó na folha da árvore especifica um valor a ser retornado pela função (RUSSEL; NORVIG, 2009).

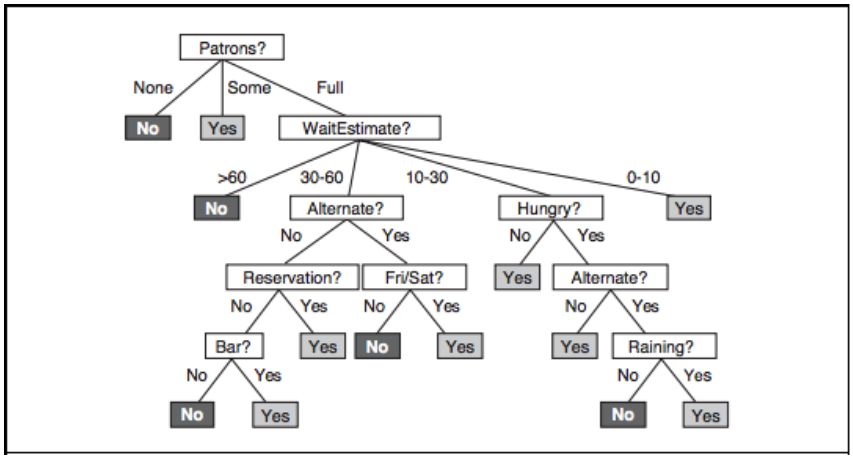
Figura 8 – Árvore de Decisão - Entrada de Atributos

Example	Input Attributes										Goal
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*WillWait*
$\mathbf{x}_1$	*Yes*	*No*	*No*	*Yes*	*Some*	*$$$*	*No*	*Yes*	*French*	*0–10*	$y_1 = Yes$
$\mathbf{x}_2$	*Yes*	*No*	*No*	*Yes*	*Full*	*$*	*No*	*No*	*Thai*	*30–60*	$y_2 = No$
$\mathbf{x}_3$	*No*	*Yes*	*No*	*No*	*Some*	*$*	*No*	*No*	*Burger*	*0–10*	$y_3 = Yes$
$\mathbf{x}_4$	*Yes*	*No*	*Yes*	*Yes*	*Full*	*$*	*Yes*	*No*	*Thai*	*10–30*	$y_4 = Yes$
$\mathbf{x}_5$	*Yes*	*No*	*Yes*	*No*	*Full*	*$$$*	*No*	*Yes*	*French*	*>60*	$y_5 = No$
$\mathbf{x}_6$	*No*	*Yes*	*No*	*Yes*	*Some*	*$$*	*Yes*	*Yes*	*Italian*	*0–10*	$y_6 = Yes$
$\mathbf{x}_7$	*No*	*Yes*	*No*	*No*	*None*	*$*	*Yes*	*No*	*Burger*	*0–10*	$y_7 = No$
$\mathbf{x}_8$	*No*	*No*	*No*	*Yes*	*Some*	*$$*	*Yes*	*Yes*	*Thai*	*0–10*	$y_8 = Yes$
$\mathbf{x}_9$	*No*	*Yes*	*Yes*	*No*	*Full*	*$*	*Yes*	*No*	*Burger*	*>60*	$y_9 = No$
$\mathbf{x}_{10}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*$$$*	*No*	*Yes*	*Italian*	*10–30*	$y_{10} = No$
$\mathbf{x}_{11}$	*No*	*No*	*No*	*No*	*None*	*$*	*No*	*No*	*Thai*	*0–10*	$y_{11} = No$
$\mathbf{x}_{12}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*$*	*No*	*No*	*Burger*	*30–60*	$y_{12} = Yes$

Figure 18.3 Examples for the restaurant domain.

Fonte: Artificial Intelligence A Modern Approach Third Edition, RUSSEL; NORVIG, 2009

Figura 9 – Árvore de Decisão - Restaurante

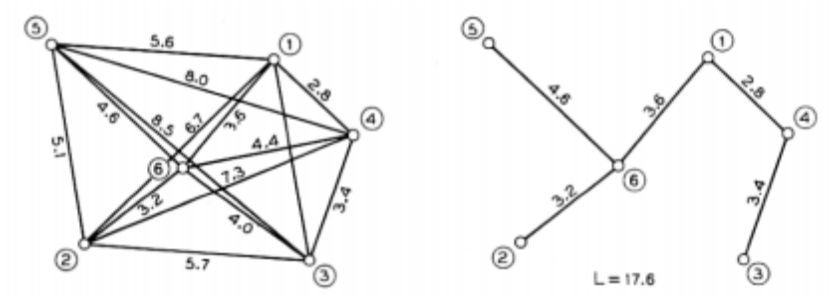


Fonte: Artificial Intelligence A Modern Approach Third Edition, RUSSEL; NORVIG, 2009

3.10.2 Algoritmo de Prim

Dado um grafo não-orientado conexo G com custos nas arestas, o algoritmo de Prim cultiva uma subárvore de G até que ela se torne uma árvore de tamanho mínimo (MST - Minimum Spanning Tree), conforme figura 10. Cada iteração começa com uma subárvore T. No início da primeira iteração, T consiste em um único vértice, escolha uma aresta que tenha custo mínimo, seja x-y a aresta escolhida, com x em T, acrescente a aresta x-y e o vértice y a T. Como se vê, o algoritmo tem caráter guloso, em cada iteração adiciona a aresta mais barata sem se preocupar com o efeito global (FEOFILOFF, 2019).

Figura 10 – Algoritmo de Prim



Fonte: Shortest Connection Networks And Some Generalizations, PRIM, 1957

3.11 Trabalhos Relacionados

Nesta seção serão tratados estudos de Inteligência Artificial em Redes Definidas por *Software*.

O trabalho *A Hybrid Intelligent Approach for Optimising Software-Defined Networks Performance* utiliza duas Inteligências Artificiais para melhorar a performance da Rede Definida por *Software*. O trabalho *Machine Learning in Software Defined Networks: Data Collection and Traffic Classification* efetua a coleta dos dados e utiliza Aprendizado de Máquina em conjunto com Redes Neurais. O trabalho *Including Artificial Intelligence in a Routing Protocol Using Software Defined Networks* utiliza agentes para definir as rotas que serão utilizadas.

3.11.1 *A Hybrid Intelligent Approach for Optimising Software-Defined Networks Performance*

A Hybrid Intelligent Approach for Optimising Software-Defined Networks Performance (SABEEH, 2016) apresenta uma nova abordagem com Inteligência Artificial Híbrida para otimizar o desempenho de Redes Definidas por *Software*, baseados em métodos de otimização heurística integrados com Rede Neural e técnicas de Otimização Evolutiva, como Otimização de Enxame de Partículas e Algoritmo Genético.

Esse projeto busca compreender qual o melhor método para fornecer o máximo de uma Rede Definida por *Software*.

Uma Rede Neural Artificial foi treinada para mapear as entradas e o desempenho da rede que em conjunto com os algoritmos evolutivos deu um resultado muito aceitável, com um erro quadrático médio inferior a 2,466 x 10-8. Pelos resultados obtidos Otimização de Enxame de Partículas supera o Algoritmo Genético (SABEEH, 2016).

3.11.2 *Machine Learning in Software Defined Networks: Data Collection and Traffic Classification*

Machine Learning in Software Defined Networks: Data Collection and Traffic Classification (AMARAL, 2016) apresenta uma arquitetura simples implantada em uma rede corporativa que reúne dados de tráfego usando o protocolo *OpenFlow*. Exibem os dados e as várias técnicas de Aprendizado de Máquina podem ser aplicadas para classificar o tráfego.

Como resultado, aprendizado supervisionado pode ser utilizado com a Rede Definida por *Software* e com os dados coletados obtiveram alto nível de precisão. Sendo possível utilizar a classificação de tráfego em campus pequenos e redes corporativas (AMARAL, 2016).

3.11.3 *Including Artificial Intelligence in a Routing Protocol Using Software Defined Networks*

Including Artificial Intelligence in a Routing Protocol Using Software Defined Networks (SENDRA, 2017) apresenta uma nova proposta para implementar um protocolo de roteamento inteligente em uma topologia de Rede Definida por *Software*, este protocolo é baseado no processo de aprendizagem de reforço que permite escolher os melhores caminhos para envio dos dados de acordo com os melhores critérios e estado da rede.

Criado um algoritmo inteligente baseado em reforço de aprendizagem, este foi adicionado ao protocolo de roteamento OSPF (*Open Shortest Path First*), com a alteração, o novo protocolo tem melhores recursos de QoS (*Quality of Service*) que os protocolos de roteamento tradicionais. Com a rede mais estável, com menos taxa de perda se tem menor atraso (SENDRA, 2017).

4 Desenvolvimento

Neste capítulo serão descritas a arquitetura, os protocolos e o Agente Inteligente utilizados neste projeto.

4.1 Arquitetura do Sistema

Para obter um melhor desempenho na Rede Definida por *Software*, foi utilizado um Agente Inteligente responsável por decidir qual é a melhor rota para enviar o fluxo de dados.

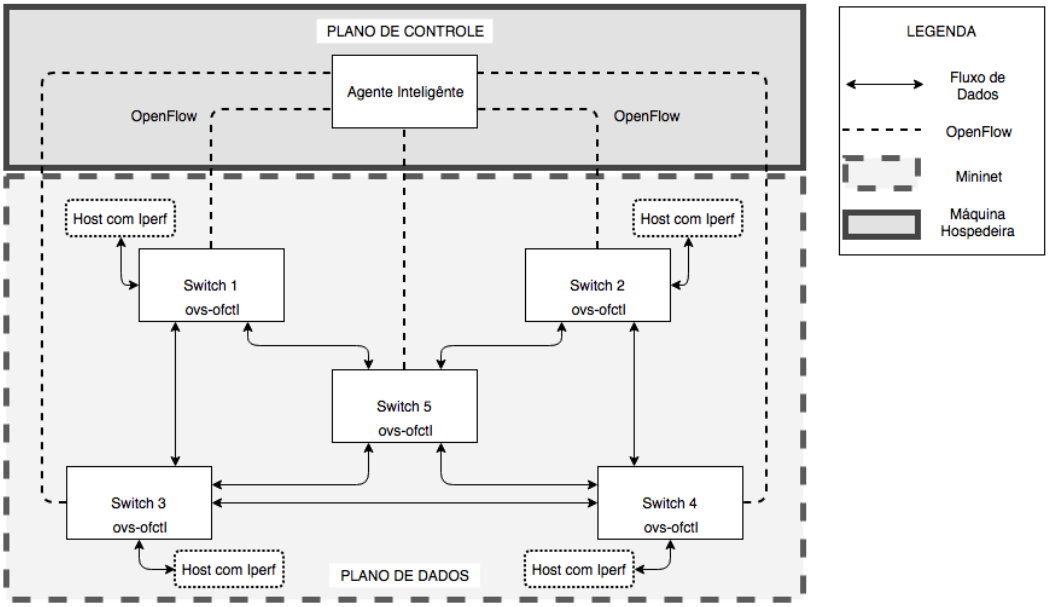
A Rede Definida por *Software* é simulada no Mininet, e foi utilizado o protocolo *OpenFlow* para se comunicar com os dispositivos no plano de dados, *Open vSwitches*, estes são configuravéis e gerenciavéis. O Agente Inteligente faz a coleta dos dados da rede pela API do Mininet.

O Agente Inteligente foi criado em *Python*, com as seguintes bibliotecas:

- pandas, para ler o arquivos csv.
- sklearn, para utilizar a árvore de decisão e validação cruzada.
- os, para efetuar comando na linha de comando do sistema operacional.
- networkx, para utilizar grafo

Na figura 11 é possível observar a arquitetura geral e o Agente Inteligente com acesso aos cinco *Open vSwitch* e ao Mininet para coletar os dados, que estão em funcionamento com rotas redundantes. Assim caso o Agente Inteligente decida, devido a algum problema, é possível alterar a rota e utilizar outro caminho.

Figura 11 – Topologia Proposta



Fonte: Criado pelo Autor

Os *hosts* utilizam o Iperf para captura de informações como perda de pacotes, *jitter*, latência e banda disponível na rede, essas informações posteriormente serão analisadas pelo agente inteligente. Existem dois grupos de *hosts*, os que executam o Iperf como servidor e os que executam como cliente, os dois grupos estão presentes em todos os *switches* que gerenciam uma rede, ou seja que atuam como *gateway* da rede.

O Agente Inteligente precisa dos dados perceptidos do seu ambiente, pra coletar os dados dos *hosts* é utilizado a API do Mininet, assim sendo capaz de verificar o estado atual da rede e verificar as mudanças que ocorreram no ambiente. Após avaliar o estado da rede, é possível verificar as consequências de suas ações e definir a melhor ação para se atingir a meta, neste projeto as metas são: utilizar rotas que reduzam a latência, a largura de banda, a perda de pacote,e o *jitter*, assim permitindo que seja mantido o fluxo contínuo dos dados.

Cada *switch* tem três tipos de rotas:

1. Entre *hosts* da mesma rede;
2. Entre *switches* vizinhos;
3. Entre *Switches* não vizinhos.

No primeiro e no segundo caso o fluxo dos dados permanecerão fixos. Porém, no terceiro caso as rotas são flexíveis e será necessário escolher uma rota de saída para passagem do

fluxo de dados.

Ao receber as informações do ambiente após a execução do Iperf, os dados passam por uma árvore de decisão que classifica cada rota de saída do *switch*. Para a classificação cruzada foi utilizado o critério gini e o *random_state* igual a 2014, para validação o conjunto de dados foi separado em cinco partes, que utiliza quatro para aprendizado e uma para treino e repete esse procedimento 5 vezes para treinar com todo o conjunto de dados, no treinamento a taxa de acerto é de 0.9503223117532649, ou seja 95,03%.

Foi criado um arquivo de teste com 300 exemplos para ser o conjunto de dados, estes definidos com vários cenários, o conjunto de dados é baseado na rede criada no Mininet e pode variar de computador para computador, o resultado da classificação é uma verificação de cada um dos itens conforme especificado abaixo:

Resultado igual a 0 (ótimo):

- Latência < 100ms
- Perda de pacote < 1
- Jitter < 1ms
- Banda > 343kb

Resultado igual a 1 (bom):

- Latência < 200ms
- Perda de pacote < 2
- Jitter < 2ms
- Banda > 200kb

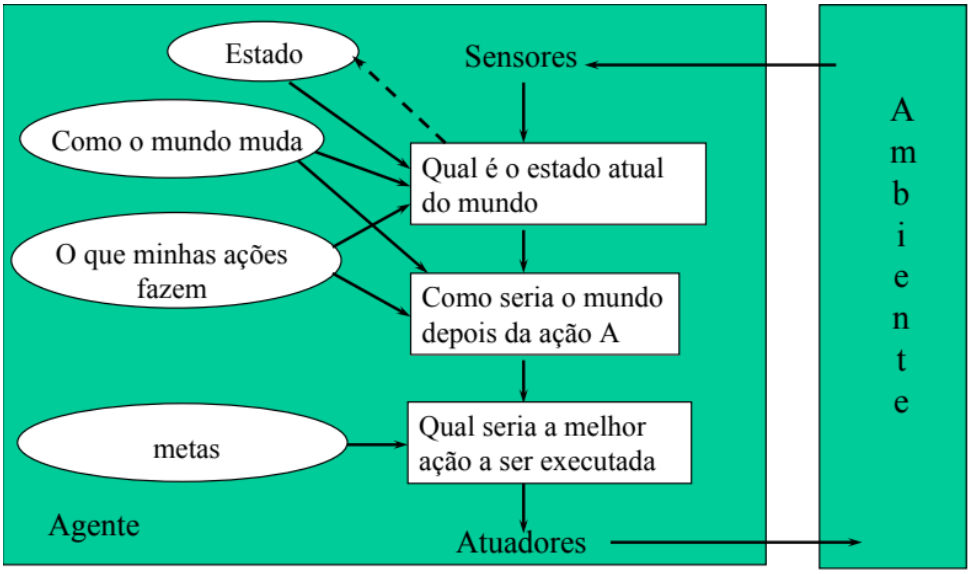
Resultado igual a 2 (ruim):

- Latência > 200ms
- Perda de pacote > 2
- Jitter > 2ms
- Banda < 200kb

Após a classifição temos um grafo não-direcionado ponderado onde é utilizado o Algoritmo de Prim para criar uma árvore de distribuição mínima, esse processo é muito importante para não criação de *loop* na rede, assim criando a comunicação entre todos os pontos da rede.

O Agente Inteligente é responsável pela alteração na tabela de fluxos do *Open vSwitch* e avalia possíveis alterações em um ou mais *Open vSwitch*, a estrutura do Agente segue como na figura 12

Figura 12 – Agente Baseado em Metas



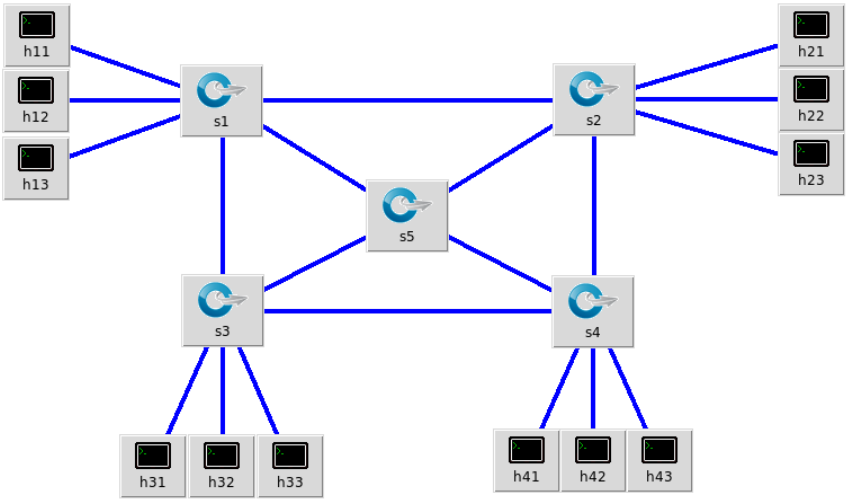
Fonte: IME-USP, Introdução à Inteligência Artificial Agentes Inteligentes, BARROS, 2006

Neste projeto as características do ambiente são:

- Completamente Observável, coletamos informações para saber o estado completo da rede;
- Não-Determinístico, o próximo estado não depende apenas do estado atual e ações do Agente;
- Dinâmico, o ambiente muda enquanto o Agente processa a ação;
- Discreto, possui um número limitado de percepções e ações.

Foi construída a topologia proposta na figura 11 no Mininet com o editor gráfico MiniEdit, conforme figura 13:

Figura 13 – Topologia criada no MiniEdit



Fonte: Criado pelo Autor

Ao construir a topologia é criado um código em *Python* com as informações do MiniEdit contendo as configurações de IP, conexões de *link* e nome dos dispositivos.

No *Mininet*, é possível com o comando *dump* verificar que os IP's foram atribuídos aos *hosts* e que foram conectados *links* entre os dispositivos conforme figura 14:

Figura 14 – Comando Dump do Mininet

```
mininet> dump
<Host h41: h41-eth0:192.168.40.41 pid=3786>
<Host h12: h12-eth0:192.168.10.12 pid=3789>
<Host h23: h23-eth0:192.168.20.23 pid=3802>
<Host h43: h43-eth0:192.168.40.43 pid=3805>
<Host h22: h22-eth0:192.168.20.22 pid=3808>
<Host h33: h33-eth0:192.168.30.33 pid=3815>
<Host h31: h31-eth0:192.168.30.31 pid=3822>
<Host h42: h42-eth0:192.168.40.42 pid=3825>
<Host h32: h32-eth0:192.168.30.32 pid=3828>
<Host h21: h21-eth0:192.168.20.21 pid=3831>
<Host h11: h11-eth0:192.168.10.11 pid=3834>
<Host h13: h13-eth0:192.168.10.13 pid=3841>
<customOvs s3: lo:127.0.0.1,s3-eth1:None,s3-eth2:None,s3-eth3:None,s3-eth4:None,s3-eth5:None,s3-eth6:None pid=3798>
<customOvs s2: lo:127.0.0.1,s2-eth1:None,s2-eth2:None,s2-eth3:None,s2-eth4:None,s2-eth5:None,s2-eth6:None pid=3811>
<customOvs s4: lo:127.0.0.1,s4-eth1:None,s4-eth2:None,s4-eth3:None,s4-eth4:None,s4-eth5:None,s4-eth6:None pid=3818>
<customOvs s5: lo:127.0.0.1,s5-eth1:None,s5-eth2:None,s5-eth3:None,s5-eth4:None pid=3837>
<customOvs s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None,s1-eth3:None,s1-eth4:None,s1-eth5:None,s1-eth6:None pid=3844>
```

Fonte: Criado pelo Autor

Dispositivo	IP	Modo do Iperf
Switch1	127.0.0.1	Não se aplica
Switch2	127.0.0.1	Não se aplica
Switch3	127.0.0.1	Não se aplica
Switch4	127.0.0.1	Não se aplica
Switch5	127.0.0.1	Não se aplica
Host11	192.168.10.11	Cliente
Host12	192.168.10.12	Cliente
Host13	192.168.10.13	Servidor
Host21	192.168.20.21	Cliente
Host22	192.168.20.22	Cliente
Host23	192.168.20.23	Servidor
Host31	192.168.30.31	Cliente
Host32	192.168.30.32	Cliente
Host33	192.168.30.33	Servidor
Host41	192.168.40.41	Cliente
Host42	192.168.40.42	Cliente
Host43	192.168.40.43	Servidor

Ao iniciar uma sessão SSH com a máquina do Mininet (IP 192.168.56.102, usuário:mininet e senha: mininet) é iniciada a topologia com o MiniEdit. Conforme figura 15:

Figura 15 – Acessando o mininet por SSH e executando o MiniEdit

Fonte: Criado pelo Autor

A topologia foi criada utilizando uma máquinas virtual e a hospedeira. Sendo essas as especificações:

Máquina virtual executando o Mininet:

- 1Gb de RAM
- 8Gb de Disco
- Processador 1x Core

- 1x Placa de rede com NAT e acesso a internet para baixar o Mininet
- 1x Placa de rede Host-Only para comunicação com IP 192.168.56.102

Máquina Hospedeira executando o Agente Inteligente:

- 8Gb de RAM, 1333 MHz DDR3
- 1Tb de Disco (SSD)
- Processador I5 2,3GHz 4x Core
- 1x Placa de rede WiFi com acesso a internet
- 1x Placa de rede virtual para comunicação com IP 192.168.56.1

4.2 Testes

Os testes foram efetuados utilizando o utilitário de linha de comando *Traffic Control* (TC), onde se pode acrescentar latência, *jitter* e perda de pacotes de forma controlada. Assim é possível efetuar a comparação deste trabalho com outros métodos.

Iremos comparar este trabalho com as redes controladas por: *OpenFlow* (sem controlador), controlador com *Spanning Tree* e controlador com *OSPF*.

O monitoramento da rede será contínuo, assim poderemos verificar os cenários e analisar principalmente os itens:

- Latência
- Perda de Pacote
- Jitter
- Banda disponível

Os testes nos cenários terão o mesmo procedimento que são:

- Medição inicial sem prejuízo de desempenho;
- Adicionar a latência do *switch* s1 e coletar os dados;
- Adicionar a latência e *Jitter* do *switch* s2 e coletar os dados;
- Adicionar perda de pacotes no *s3* e coletar os dados.
- Remover qualquer prejuízo ao *switch* s1

- Remover qualquer prejuízo ao *switch* s2
- Remover qualquer prejuízo ao *switch* s3

4.2.1 Teste do Agente Inteligente

Ao iniciar o teste, todas as arestas tem peso 0, conforme tabela a seguir:

Aresta u	Aresta v	Valor
s3	s1	0
s3	s5	0
s3	s4	0
s2	s1	0
s2	s5	0
s2	s4	0
s1	s5	0
s5	s4	0

Para poder realizar a coleta dos dados é necessário ter a comunicação entre todos os *hosts* da rede. Para isso é adicionada uma regra na tabela ovs-ofctl de cada *switch* liberando a própria rede. Posteriormente, são adicionadas as regras para os *switches* vizinhos. Para comunicação com os outros é preciso adicionar rotas de forma que não gere *loop* na rede, para isso nesse momento é utilizado o Algoritmo de Prim para gerar uma árvore de distribuição mínima, com a árvore criada adicionamos as rotas conforme grafo gerado.

Com todos os *hosts* em comunicação é iniciado o *Iperf* em modo servidor nos *hosts* h13, h23, h33 e h43 com o comando "iperf -s -u", para executar esse comando no *host* h13 pela API do Mininet foi utilizado o comando: "ssh mininet@192.168.56.102 mininet/util/m h13 iperf -s -u &".

A coleta de dedos é feita dos *hosts* h11, h21, h31 e h41, cada um irá efetuar um teste com cada um dos *hosts* executando o Iperf como servidor, nesse exemplo tem o *host* h11 verificando o *host* h23 com os comandos:

- Latência: ssh mininet@192.168.56.102 mininet/util/m h11 ping -c 1 h23 |grep icmp | awk {'print $7'} | cut -d= -f2 | cut -d. -f1
- Perda de Pacote: ssh mininet@192.168.56.102 mininet/util/m h11 iperf -c h23 -u |grep '%' | awk {'print $NF'} |cut -d'%' -f1 | cut -d'(' -f2
- *Jitter*: ssh mininet@192.168.56.102 mininet/util/m h11 iperf -c h23 -u | grep '%' |awk {'print $9'}
- Banda disponível: ssh mininet@192.168.56.102 mininet/util/m h11 iperf -c h23 -u -f -k |grep '%' | awk {'print $7'}

Como a comunicação não há *loops* então só são verificadas as rotas ativas.

Com os valores é formado um vetor que é verificado pela árvore de decisão treinada com o arquivo de exemplo e assim define o desempenho da aresta. Se for igual a 0, será reduzido em 10 o peso da aresta. Se for igual a 1 não terá alteração no peso. Se for igual a 2, será adicionado em 100 o peso da aresta.

Após o ajuste nos pesos das arestas é gerado um grafo ponderado não direcionado, para eliminar o *loop* na rede foi utilizado o Algoritmo de Prim, reduzindo o número de arestas e gerando uma árvore de distribuição mínima.

O Agente Inteligente verifica as arestas do grafo e utiliza a API do *Mininet* para informar aos *switches* qual placa deverá utilizar para chegar em um *switch* que não é vizinho.

Caso tenha alteração de desempenho na rede, após esse procedimento poderá ter alteração nas rotas dos *switches*, o *switch* s5 não terá uma rota de saída, pois não é *gateway* de nenhuma rede e todos os *switches* são vizinhos.

A cada teste entre o *host* que executa o *Iperf* como cliente com um *host* com *Iperf* como servidor são efetuados todos os passos citados anteriormente.

Ao iniciar o procedimento de teste, a rede não tem prejuízo no desempenho, dessa forma em primeira análise o retorno é: Latência igual a 0, perda de pacote igual a 0, jitter igual a 0.042 e banda disponível de 341.

Inicialmente as rotas eram:

switch s3: sai pelo *switch* s4

switch s2: sai pelo *switch* s1

switch s1: sai pelo *switch* s5

switch s4: sai pelo *switch*: s3

Ao adicionar latência de 180ms na aresta entre o *switch* s1 e o *switch* s3 não é suficiente para deixar , as rotas permaneceram iguais.

Ao adicionar latência de 180ms e 5ms de *jitter* na aresta entre o *switch* s2 e o *switch* s1 em alguns casos que passam pelo *switch* s1 e s2 a latência chegou a 360ms e começaram a aparecer as primeiras rotas com classificação 2. As rotas foram alteradas para:

switch s3: sai pelo *switch* s1

switch s2: sai pelo *switch* s4

switch s1: sai pelo *switch* s5

switch s4: sai pelo *switch*: s5

Os pesos que iniciaram todos com 0, nesse momento estavam em:

switch s3 e s1: 100

switch s3 e s5: 0

switch s3 e s4: 90

switch s2 e s1: 90

switch s2 e s5: 90

switch s2 e s4: 0

switch s1 e s5: 100

switch s5 e s4: 100

Todos os *switches* tentaram se ajustar. Antes de acrescentar 50% de perda de pacotes na aresta entre o *switch* s3 e o *switch* s1 tinham o seguinte peso:

switch s3 e s1: 180

switch s3 e s5: 100

switch s3 e s4: 150

switch s2 e s1: 180

switch s2 e s5: 180

switch s2 e s4: 180

switch s1 e s5: 100

switch s5 e s4: 180

Novamente todos os *switches* tentaram se ajustar. Antes de remover os prejuízos nos *switches* s1 e s2, os pesos estavam em:

switch s3 e s1: 360

switch s3 e s5: 280

switch s3 e s4: 510

switch s2 e s1: 450

switch s2 e s5: 360

switch s2 e s4: 370

switch s1 e s5: 370

switch s5 e s4: 360

É possível perceber que algumas arestas não tiveram alteração e uma melhorou, isso ocorre, pois o prejuízo no desempenho estava apenas no *switch* s3, antes de ajustar o

switch s3 os pesos estavam em:

switch s3 e s1: 540

switch s3 e s5: 460

switch s3 e s4: 510

switch s2 e s1: 450

switch s2 e s5: 450

switch s2 e s4: 500

switch s1 e s5: 330

switch s5 e s4: 450

Agora sem nenhum prejuízo na rede os pesos, houve alguns casos de perda de pacotes no início, mas entre a coleta 22 e 30 teve a seguinte alteração:

switch s3 e s1: de 540 para 520

switch s3 e s5: de 550 para 550

switch s3 e s4: de 510 para 570

switch s2 e s1: de 540 para 540

switch s2 e s5: de 450 para 540

switch s2 e s4: de 500 para 530

switch s1 e s5: de 330 para 580

switch s5 e s4: de 450 para 630

4.2.2 Teste Controlador OpenDayLight

Nesse cenário foi utilizada uma segunda máquina virtual com IP 192.168.54.101, com a instalação do *OpenDayLight* e as *features*: odl-restconf, odl-mdsal-apidocs, odl-dlux-all, odl-l2switch-switch e odl-l2switch-switch. Onde os dois últimos são responsáveis pela comunicação entre *switches*.

No Mininet foram removidas todas as rotas e apontado o controlador para o IP 192.168.54.101.

Ao iniciar não há comunicação, mas o *switch* consulta o controlador a cada pacote que o não sabe por onde enviar, em pouco tempo há comunicação em toda a rede.

Sem prejuízo no desempenho da rede, a primeira coleta tem os valores:

ping: 0 ms drop: 0 jitter: 0.01ms banda disponível: 343kb

Quando adicionado 180ms de latência na comunicação entre o *switch* s1 e o *switch*

s3 não há mudança de rotas na rede, a coleta retorna em rotas influenciadas pelo acréscimo com os valores:

ping: 180ms drop: 0 jitter: 0.136ms banda disponível: 339kb

Quando adicionado 180ms de latência e 5ms de *Jitter* na comunicação entre o *switch* s2 e o *switch* s1 a latência por volta de 360ms, pois algumas rotas passam tanto pelo *switch* s1 como pelo *switch* s2, e há casos de perda de pacotes em todos os testes, mesmo sem adicionar o prejuízo de perda de pacote em nenhum *switch*. Segue os valores em dois casos:

ping: 357ms drop: 0 jitter: 0.081 banda: 1.05

ping: sem resposta drop: sem resposta jitter: sem resposta banda: sem resposta

Ao adicionar perda de pacote de 50% entre o *switch* s3 e o *switch* s1 houve perda de pacotes em todos os casos.

Ao remover o prejuízo de desempenho nos *switches* s1 e s2 normalizaram as rotas, com exceção quando passava no *switch* s3 que ainda perdia pacote.

Ao remover o prejuízo de desempenho do *switch* s3, não houve alteração na perda de pacotes. Ninguém conseguia chegar na rede do *switch* s3 e nem a rede do *switch* s3 conseguia chegar em outra rede.

4.3 Avaliação e conclusões

O Agente Inteligente criado consegue extrair os dados utilizando o *Iperf* e percebe quando o desempenho diminui, assim altera as rotas em busca de melhor desempenho. Com a árvore de decisão utilizada para classificação e o Algoritmo de Prim consegue gerar caminhos que não geram *loops* e mantém a rede em comunicação mesmo em casos de baixo desempenho.

O Controlador *OpenDayLight* cria rapidamente as rotas, mas quando tem alteração de desempenho as rotas não se alteram, criando falha na comunicação. Nesse cenário haveria uma queda de desempenho que precisaria de ajuste manual.

4.3.1 Trabalhos Futuros

Alguns trabalhos futuros possíveis são:

- Utilizar serviços de redes como *e-mail*, *VoIP* entre outros para classificação das rotas;
- Utilizar dispositivos *wireless* para comutação;
- Utilizar outras variáveis para classificação com foco no plano de negócio pré-definido.

4.4 Conclusão

Neste trabalho foi configurado um Agente Inteligente que consegue por API gerenciar e coletar dados de uma Rede Definida por *software* criada no *Mininet*, contendo *hosts* e *switches* (com *Open vSwitch*) com Comunicação via *OpenFlow*.

Com a coleta dos dados pelo *Iperf* foi possível verificar o estado da rede e utilizar uma árvore de decisão para classificar os dados e em conjunto com o Algoritmo de Prim foi possível estabelecer comunicação entre todos os hosts sem gerar *loops* nas rotas, assim o Agente Inteligente pôde alterar as rotas para escolher a que tem o melhor desempenho e manter o fluxo dos dados.

A rede é vista pelo Agente Inteligente como um grafo não-direcionado ponderado. Quando alguma rota tem alguma queda de desempenho o Agente Inteligente adiciona um peso na aresta, assim é possível utilizar outra rota com menor peso.

Nos testes com o Agente Inteligente é possível acompanhar a alteração das rotas e a busca pelo melhor caminho, o que não ocorre quando se retira o Agente Inteligente e adiciona o controlador *OpenDayLight*, pois quando a rede é iniciada pelo controlador são definidas rotas que não sofrem alteração mesmo que tenha uma grande perda de desempenho.

Nesse cenário de perda de desempenho o Agente Inteligente teve um resultado melhor que o controlador, porém, o controlador é aberto a API e é possível adicionar algum outro sistema de roteamento que o melhore.

5 Referências

AMARAL, A. et al. Machine Learning in Software Defined Networks: Data Collection and Traffic Classification. IEEE 24th International Conference on Network Protocols (ICMP). 2016.

APPLEBY, D. S.; CALLAHAN, Dale. A Documentation Framework for Computer Network Design Reviews. University of Alabama at Birmingham. IEEE, 2002.

BARROS, L. N. Introdução à Inteligência Artificial Agentes Inteligentes. IME-USP, 2006.

CONTERATO, M. et al. Avaliação do suporte à simulação de redes OpenFlow no NS-3. Porto Alegre. 2013.

DAUTOVIC, Andreas. Et al. The Value of Software Documentation Quality. Johannes Kepler University. IEEE, 2008.

DEBIAN. Iperf(1). Disponível em: <https://manpages.debian.org/testing/iperf/iperf.1.en.html Acesso em: 02 dez. 2019.

DEBIAN. tc(8). Disponível em: <https://manpages.debian.org/stretch/iproute2/tc.8.en.html> Acesso em: 02 dez. 2019.

FAUSTINI, Pedro Henrique Arruda. *Emprego de NFV e aprendizagem de máquina para detectar e mitigar anomalias em redes definidas por software*. 2016. 56f. Monografia de TCC - Universidade Federal do Rio Grande do Sul, Porto Alegre, 2016.

FAQ MININET. MININET TEAM. Disponível em: <github.com/mininet/mininet/wiki/FAQ#remote-control >. Acesso em: 29 abr. 2019

FEOFILOFF, P. Algoritmo de Prim. IME-USP, 2006. Disponível em: <https://www.ime.usp.br/ goritmos_para_grafos/aulas/prim.html> Acesso em: 18 nov. 2019

JENNINGS, N. R. et al. A Roadmap of Agent Research and Development. Kluwer Academic Publishers. 1998.

JUNIOR, L. V. REDES DEFINIDAS POR SOFTWARE: Um guia compilado sobre a evolução das redes, data centers e suas novas arquiteturas, tecnologias e paradigmas. 2014.

KREUTZ, D. Software-Defined Networking: A Comprehensive Survey. Proceedings of the IEEE, Vol. 103, No. 1, Janeiro 2015

LIMA, E. S. INF 1771 – Inteligência Artificial: Aula 02 – Agentes Inteligentes. PUC-RIO, 2012.

MININET. MININET TEAM. Disponível em: <mininet.org>. Acesso em: 03 mar. 2018.

NADEAU, T. D.; GRAY, K. SDN - Software Defined Networks. O'REILLY, 2013

ONF (Open Networking Foundation). OpenFlow Switch Specification. 2009.

OpenSSH. ssh. Disponível em: <https://man.openbsd.org/ssh>. Acesso em: 02 dez. 2019

Our Mission. OPENDAYLIGHT. Disponível em: <https://www.opendaylight.org>. Acesso em: 23 fev. 2019

PRIM, R. C. Shortest Connection Networks And Some Generalizations, 1957.

RUSSEL, S.; NORVIG, P. Artificial Intelligence A Modern Approach Third Edition. Prentice Hall. 2009.

SABEEH, A. et al. Hybrid Intelligent Approach for Optimising Software-Defined Networks Performance. 6th International Conference on Information Communication and Management. 2016.

SENDRA, S. et al. Including Artificial Intelligence in a Routing Protocol Using Software Defined Networks. 5th IEEE International Workshop on Smart Communication Protocols and Algorithms (SCPA). 2017.

SISOV, Maksim. Building a Software-Defined Networking System with OpenDaylight Controller. Helsinki Metropolia University of Applied Sciences. 2016.

TSATSOULIS, C.; SOH, L. Intelligent Agents in Telecommunication Networks. CRC Press. 2000.

XIAO, Debao. Et al. Human Body Network: Network in the Future?. Institute of Computer Network & Communication Technology CCNU. IEEE, 2008.

WHAT IS AN OPENFLOW CONTROLLER?. SDX CENTRAL. Disponível em: <www.sdxcentral.com/sdn/definitions/sdn-controllers/openflow-controller/>. Acesso em: 14 abr. 2018.

WHAT IS OPEN VSWITCH (OSV). SDX CENTRAL. Disponível em: <www.sdx central.com/cloud/open-source/definitions/what-is-open-vswitch>. Acesso em: 19 mai. 2018.